

Ch10 练习：数组和指针

重点章 · 6 题：填空 1 → 改错 1 → 编程 3 → 专业场景拓展 1 + 复习题索引（索引与参考答案区不计题数）。改错题诊断均为本机 `gcc -Wall -Wextra 16.1.0` 实测。

范围自律：不考指针高级用法（函数指针/指针数组/多级指针）。编译命令：`gcc -Wall -Wextra -o prog.exe prog.c`

10-1 填空（★）

填空 ★

补全代码，使注释成立：

```
int a[4] = {5, 10, 15, 20};

printf("%d", ____);    /* 输出：15 */
int *p = ____;         /* p 指向数组开头 */
printf("%d", *(p+1));  /* 输出：____ */
printf("%d", a[4]);     /* 危险！这属于____（越界/合法访问）*/
```

10-2 改错（★★，先自己找再对答案）

改错 ★★

下面程序有 **4 处问题**，找出并改正，分类为「编译期报错 / 仅警告 / 静默逻辑错」：

```
#include <stdio.h>

int main(void)
{
    int a[3] = {1, 2, 3};
    int *p;           /* ① */
    *p = 10;
    printf("地址 %d\n", &a[0]); /* ② */
    printf("a[3] = %d\n", a[3]); /* ③ */
    int *q = 100;      /* ④ */
    return 0;
}
```

提示：野指针 / 格式串类型不匹配 / 下标越界 / 整数赋给指针。

10-3 编程 (★★)

编程 ★★

写程序：读入 5 个整数存进数组，输出**最大值**和**平均值**（保留 1 位小数）。

输出样例：

```
请输入 5 个整数：3 8 1 9 4
最大值 = 9  平均值 = 5.0
```

10-4 编程 (★★★)

编程 ★★★

写程序：用**指针**遍历数组（不许用下标），把数组 {4, 2, 9, 7, 1} 逆序打印。

要求：至少用到 `p++` 和 `*p`。

输出样例：

```
1 7 9 2 4
```

10-5 编程 (★★★)

编程 ★★★

写程序：定义函数 `void fill(int arr[], int n, int v)`，把数组全部元素设成 `v`，然后在 `main` 里声明 `int a[6]` 调用它填成 7 并打印。验证函数里改动确实影响外面。

10-6 专业场景拓展 (★★★，不需要硬件知识)

拓展 ★★★

ADC 采样场景：传感器 1 秒采样 10 次，值范围 0-1023。用数组存 10 个采样值（自定数据），写程序找**最大值对应的下标**并输出「第 N 次采样最大」。（这题只练数组遍历 + 找极值，不需要硬件知识。）
输出样例：

```
采样：512 780 333 910 456 820 999 120 650 780
第 7 次采样最大 = 999
```

10-7 复习题索引

打开《C Primer Plus（第6版）》第 10 章，完成复习题 2、4、6。题号对应主教材，原题不入本册。
做完对照本书 10.4 的等价公式 `a[i] == *(a+i)` 再检查一遍。

10-8 参考答案（先自己写，再看思路与常见错法）

10-1

依次填：`a[2]`、`a`（或 `&a[0]`）、10、**越界**。

思路：下标从 0 开始，`a[2]` 是第 3 个元素；数组名就是首元素地址。

常见错法：写 `a[3]` 找 15（那是第 4 个）；以为 `a[4]` 合法（它已经出界了）。

10-2

四处问题及实测诊断（gcc 16.1.0 -Wall -Wextra 实测）：

- ① `int *p; *p = 10;` 野指针：单独编译（其余修好）会报
warning: 'p' is used uninitialized [-Wuninitialized]; 本题整段因 ④ 的硬错误
编译器未进入该分析故看不到。解引用写内存 = 运行期崩溃/UB（恰好没崩只是运气）
- ② `printf("地址 %d\n", &a[0]);` 仅警告：format '%d' expects 'int' but
argument has type 'int *' [-Wformat=]（地址要用 %p）
- ③ `a[3]` 越界 静默逻辑错：无诊断。C 不检查下标，
读到数组后面的垃圾内存
- ④ `int *q = 100;` 编译期报错：makes pointer from integer
without a cast [-Wint-conversion]

改法：① `int *p = &a[0];`（或先指向数组再解引用）；② `printf("地址 %p\n", (void*)&a[0]);`；③
循环边界写 `i < 3`（读 `a[0..2]`）；④ `int *q = &a[0];`。

常见错法：把「没报错」当「没问题」——②④一定报；①在隔离编译时也会被 -Wuninitialized 抓到，但整段含硬错误时可能看不到，所以**不能靠「有没有警告」判断野指针**；③越界任何时候都不报，只能靠自己的边界意识。

10-3

思路：循环读入存数组；max 初始为 `a[0]` 再逐个比；平均值 = 总和 / 5（%.1f）。

```
#include <stdio.h>
int main(void){
    int a[5], i, max, sum = 0;
    printf("请输入 5 个整数: ");
    for (i = 0; i < 5; i++) scanf("%d", &a[i]);
    max = a[0];
    for (i = 1; i < 5; i++) if (a[i] > max) max = a[i];
    for (i = 0; i < 5; i++) sum += a[i];
    printf("最大值 = %d  平均值 = %.1f\n", max, sum / 5.0);
    return 0;
}
```

常见错法：max 初始化成 0（数组可能全是负数）；sum / 5 整数除法丢小数（要 5.0）；下标写成 `<= 5` 越界。

10-4

思路：指针从数组末尾开始往前挪（`p--`），或先把 `p` 移到末尾再倒走。

```
#include <stdio.h>
int main(void){
    int a[5] = {4, 2, 9, 7, 1};
    int *p;
    for (p = a + 4; p >= a; p--)
        printf("%d ", *p);
    printf("\n");
    return 0;
}
```

常见错法：指针比较用 `p >= a` 是允许的（同一数组内）；把 `a+4` 写成 `a[4]` 就成值不是地址了。

10-5

思路：形参写 `int arr[]`，函数里 `arr[i] = v` 直接改外面（传的是地址）。

```
#include <stdio.h>
void fill(int arr[], int n, int v){
    int i;
    for (i = 0; i < n; i++) arr[i] = v;
}
int main(void){
    int a[6], i;
    fill(a, 6, 7);
    for (i = 0; i < 6; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

常见错法：以为形参数组是副本、改不动外面——这正是本章最该破除的误解。

10-6

思路：遍历数组记录「当前最大值的下标」，遇到更大的更新。

```
#include <stdio.h>
int main(void){
    int s[10] = {512, 780, 333, 910, 456, 820, 999, 120, 650, 780};
    int i, idx = 0;
    for (i = 1; i < 10; i++)
        if (s[i] > s[idx]) idx = i;
    printf("第 %d 次采样最大 = %d\n", idx + 1, s[idx]);
    return 0;
}
```

常见错法：输出 `idx` 而不 +1（人类数「第 1 次」是下标 0）；并列最大值只取第一个（若题目要求全部列出需另写）。

10-7 复习题索引：去书上做 Ch10 复习题 2、4、6（题号对应主教材，原题不入本册）。

10-9 一分钟自测答案（对应讲义 10.9）

自测 ①

正解：不会。`a[5]` 已经越界——`int a[5]` 的合法下标只有 0–4。它读到的是数组后面那块内存，值不确定（可能是别的变量，也可能是垃圾值）。

常见错解：「会，因为 `a[5]` 是第 5 个元素。」——错在把「第 5 个」当成 `a[5]`；第一个元素是 `a[0]`，第 5 个是 `a[4]`。

为什么容易错：日常语言从 1 数起，C 从 0 数起；而且越界**不报错**，写错的人常常「看起来能跑」。

自测 ②

正解：语法合法，但行为未定义（UB），不能这么写。`p` 没有初始化，里面是残留值，`*p = 10` 等于往一块不知道属于谁的内存里写。

常见错解：「编译能过就是合法。」——错在混淆「编译通过」与「行为正确」；C 里这两件事经常不一致，这也是本章反复强调「看编译器警告」的原因。

本机实测：单独编译这一段，gcc 16.1.0 报 warning: 'p' is used uninitialized [-Wuninitialized]（不开 -O2 都报）。

正确做法：先让指针有明确指向（`int n; int *p = &n;`），或指向动态内存（Ch12）。

自测 ③

正解：实际是 `int *arr`；`sizeof(arr)` 是指针大小（本机 64 位 = 8），不是整个数组。

常见错解：「是数组，`sizeof` 应该等于元素个数 $\times$ 4。」——错在把形参当成了数组本体：数组名作实参时会「退化」成指向首元素的指针，函数拿到的是地址，不是副本。

交叉验证（讲义 10.3 实验二实测）：`main` 里 `sizeof(a) = 20`，同一个数组传进函数后 `sizeof(arr) = 8`。

本练习册为自写原创变式题与自写参考讲解；改错题诊断为本机 MinGW-W64 gcc 16.1.0 实测输出。原书题号索引对应人民邮电出版社《C Primer Plus（第6版）》。